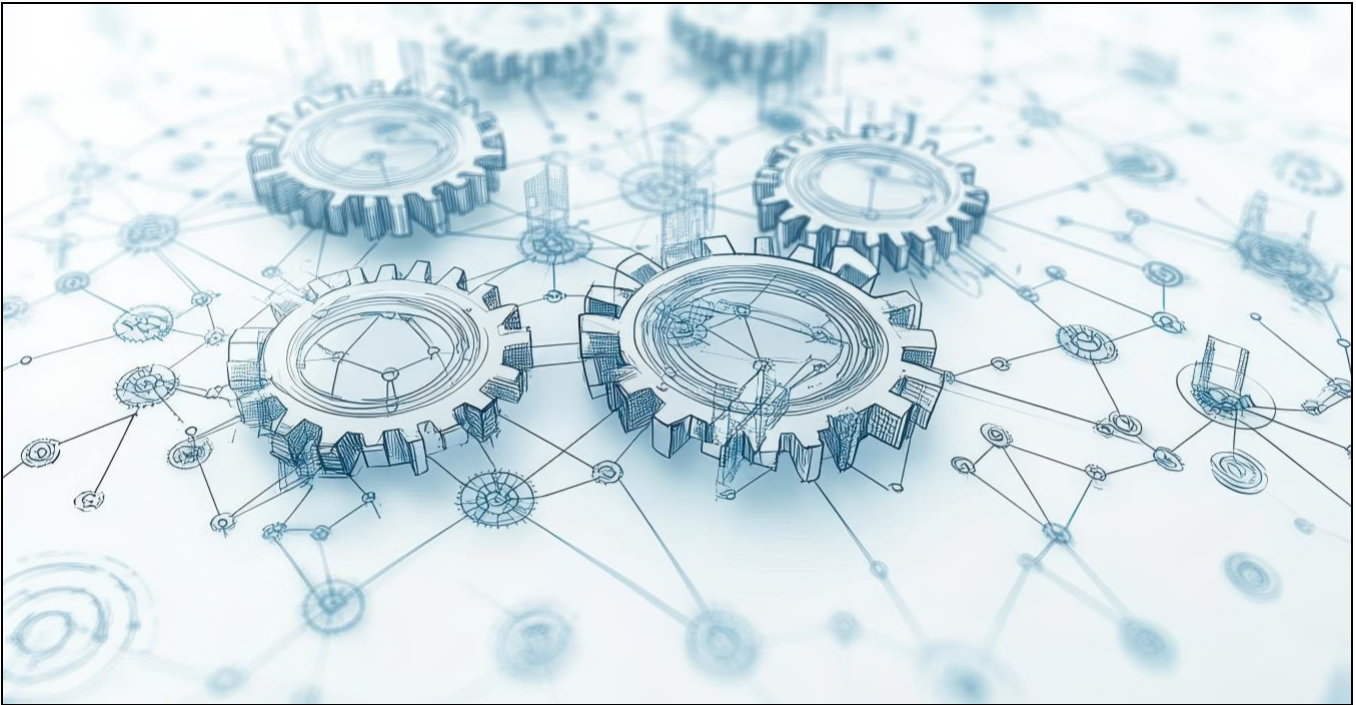


Level 4

Watsonx Orchestrate

Hands-on lab guide: Creating tools and agents



Version 3.0

February 2026

LEGAL NOTICES

This information was developed for products and services offered in the USA. IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service. IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing IBM Corporation
North Castle Drive, MD-NC119 Armonk, NY 10504-1785
United States of America

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you. This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice. Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk. IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you. Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All names and references for organizations and other business institutions used in this deliverable's scenarios are fictional. Any match with real organizations or institutions is coincidental. All names and associated information for people in this deliverable's scenarios are fictional. Any match with a real person is coincidental.

TRADEMARKS

IBM, the IBM logo, and [ibm.com](https://www.ibm.com) are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at “Copyright and trademark information” at <https://www.ibm.com/legal/copyright-trademark>.

Adobe, the Adobe logo, PostScript, and the PostScript logo are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, and/or other countries.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates. INFOBLOX is a trademark of Infoblox Inc. or its affiliated companies, registered in the United States and other countries.

Microsoft®, Azure, Windows, Windows NT, Windows Server, Microsoft® Excel®, Microsoft® Word®, Microsoft® Outlook®, Microsoft® Excel® logo, Microsoft® Word® logo, Microsoft® Outlook® logo, Microsoft® Windows App, Microsoft® Windows App logo and the Windows logo are trademarks of Microsoft® Corporation in the United States, other countries, or both.

Salesforce and Salesforce logo are registered trademarks of SALESFORCE, INC. (CORPORATION; CALIFORNIA, USA).

Slack is a registered trademark of Salesforce, Inc (CORPORATION; DELAWARE, USA).

Red Hat®, JBoss®, OpenShift®, Fedora®, Hibernate®, Ansible®, CloudForms®, RHCA®, RHCE®, RHCSA®, Ceph®, and Gluster® are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Google, Google logo, Google Tasks, Google Cloud Platform, GMAIL, GMAIL logo are registered trademarks of GOOGLE LLC and this material is not endorsed by or affiliated with Google in any way.

Elasticsearch, Elastic search logo, Elastic, Elastic logo are registered trademarks of Elasticsearch BV (LIMITED LIABILITY COMPANY; NETHERLANDS) in United States, other countries or both.

Milvus, Milvus logo are registered trademarks of LF Projects, LLC (LIMITED LIABILITY COMPANY; DELAWARE, USA) in United States, other countries or both.

Meta, Meta logos, Llama, Llama logos are registered trademarks of META PLATFORMS, INC. (CORPORATION; California, USA) in United States, other countries or both.

Mistral, Mistral logo, Mistral AI, Mistral AI logo are registered trademarks of Mistral AI (SOCIÉTÉ PAR ACTIONS SIMPLIFIÉE (SAS); FRANCE) in United States, other countries or both.

PyCharm, PyCharm logo are registered trademarks of JetBrains s.r.o. (CORPORATION; CZECH REPUBLIC) in United States, other countries or both.

Bruno is a trademark held by Anoop M D. Bruno logo is sourced from OpenMoji. License: CC BY-SA 4.0

Apple, Apple logo, Apple store, App store, App store logo are registered trademarks of Apple Inc. (CORPORATION; CALIFORNIA, USA) in United States, other countries or both.

VMWARE, VMWARE logos are registered trademarks of VMware LLC (LIMITED LIABILITY COMPANY; Delaware, USA) in United States, other countries or both.

RANCHER, RANCHER logo, Rancher Desktop, Rancher Desktop logo are registered trademarks of SUSE LLC (LIMITED LIABILITY COMPANY; MASSACHUSETTS, USA) in United States, other countries or both.

Langfuse, Langfuse logo are registered trademarks of Langfuse GmbH (GESELLSCHAFT MIT BESCHRÄNKTER HAFTUNG (GMBH); Germany, Germany) in United States, other countries or both.

OpenMeteo is a registered trademark owned by Patrick Zippenfenig (Individual, Germany) in United States, other countries or both.

© Copyright International Business Machines Corporation 2025. This document may not be reproduced in whole or in part without the prior written permission of IBM.

Table of Contents

- Introduction 1**
 - Lab overview.....1
 - What you will learn1
 - How to get support1
 - Prerequisites1
- Listing available models using the CLI..... 3**
- Creating and configuring a new agent 4**
 - Creating a tool by importing a web-service4
 - Agent creation.....8
- Listing available agents using the CLI..... 9**
- Testing the weatherman agent 10**
 - Agent and tool testing..... 10
- Summary 13**
- Next steps 14**
- Appendix: Troubleshooting..... 15**
 - System unresponsive 15
 - LLM Error 15
 - Weatherman error 15
- Appendix: Reset your environment 16**

Introduction

Lab overview

This lab will show you how to create, configure, deploy and run agents that use tools, using the watsonx Orchestrate Agent Development Kit (ADK 2.4).

When complete, the agent created will be able to respond to queries leveraging tools that provide the agent with live information from online sources, in this example a weather API.

You will then be able to build on that knowledge in later labs, creating more complex agents using the watsonx Orchestrate Agent Development Kit (ADK).

What you will learn

Once you have completed this lab you will have developed skills enabling you to complete the following tasks:

1. Create tools based on web services using an OpenAPI definition.
2. Create, configure and deploy an agent that uses the tool.
3. Test the agent you have created.
4. Use the CLI to discover agents available in your environment.

How to get support

If you encounter an issue with this lab,

- IBM employees can open an IBM Support ticket at <http://ibm.biz/support>.
- Business Partners can request help at the [Partner Plus Support](#) website.

Prerequisites

Make sure the following prerequisites are satisfied before proceeding with this lab:

- You have performed the Preparation Lab guide including the environment validation step
- You have your local ADK2.4 environment up and running (Orchestrate server started after validating the environment in the preparation guide OR after having performed another ADK related lab)

Note: If for some reason your server is not started correctly, reset your server following the [Reset your environment](#) appendix.

The tools created in this lab are using web-services based on the OpenAPI Specification (OAS), basic knowledge of the format is not required but it is useful to improve your own knowledge and to extend agents behavior using tools that leverage web-services.

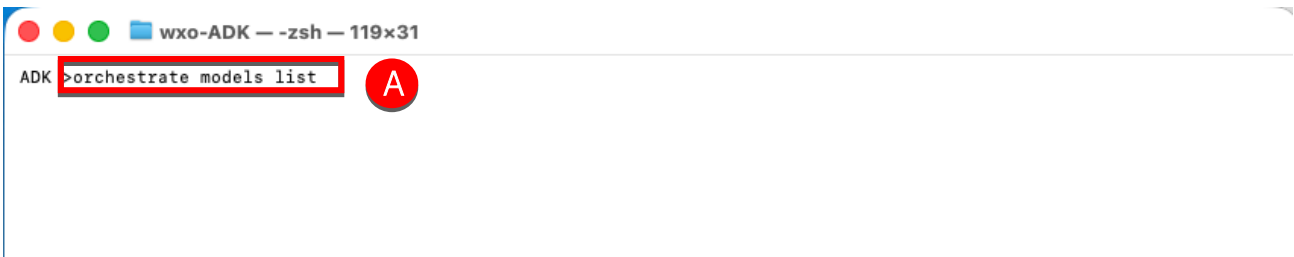
Extensive OAS resources are available online, this link from the OpenAPI initiative provides further details (<https://www.openapis.org/what-is-openapi>).

Listing available models using the CLI

In this section you will be using the CLI to view the list of LLM models available to your installation of watsonx Orchestrate ADK.

- 1. Open the ADK environment and start the watsonx Orchestrate server if not yet done. Refer to the [Reset your environment appendix](#) if your orchestrate server is not started.
- 2. To list the LLM models available with watsonx Orchestrate installation SDK run the following command (A):

```
orchestrate models list
```



- 3. The results should resemble the example below (A).

[INFO] - Retrieving virtual-policies models list...

[INFO] - Retrieving watsonx.ai models list...

Available Models	
Model	Description
* bedrock/openai.gpt-oss-120b-1:0	openai/gpt-oss-120b is an OpenAI's open-weight models designed for powerful reasoning, agentic tasks, and versatile developer use cases.
* groq/openai/gpt-oss-120b	openai/gpt-oss-120b is an OpenAI's open-weight models designed for powerful reasoning, agentic tasks, and versatile developer use cases.
* watsonx/meta-llama/llama-3-2-90b-vision-instruct	Llama-3-2-90b-vision-instruct is an auto-regressive language model that uses an optimized transformer architecture.
* watsonx/meta-llama/llama-3-405b-instruct	Llama-3-405b-instruct is Meta's largest open-sourced foundation model to date, with 405 billion parameters, optimized for dialogue use cases
watsonx/ibm/granite-3-1-8b-base	Granite 3.1 8b base is a pre-trained autoregressive foundation model with a context length of 128k intended for tuning.
watsonx/ibm/granite-3-2-8b-instruct	Granite 3.2 8b Instruct is a text-only model capable of reasoning which can be enabled or disabled by the user to use the right capability for the right use case.
watsonx/ibm/granite-3-3-8b-instruct	Granite-3.3-8b-Instruct is an IBM-trained, dense decoder-only models, which is particularly well-suited for generative tasks.
watsonx/ibm/granite-3-8b-instruct	The Granite model series is a family of IBM-trained, dense decoder-only models, which are particularly well-suited for generative tasks.

This output shows all the LLM models that are available in your installation and ready to be used with the agents we will create later.

Each available model identifier is listed on the left column and on the right, there is a description of the model and its characteristics.

Notice that a * mark in front of a model indicates supported and preferred models.

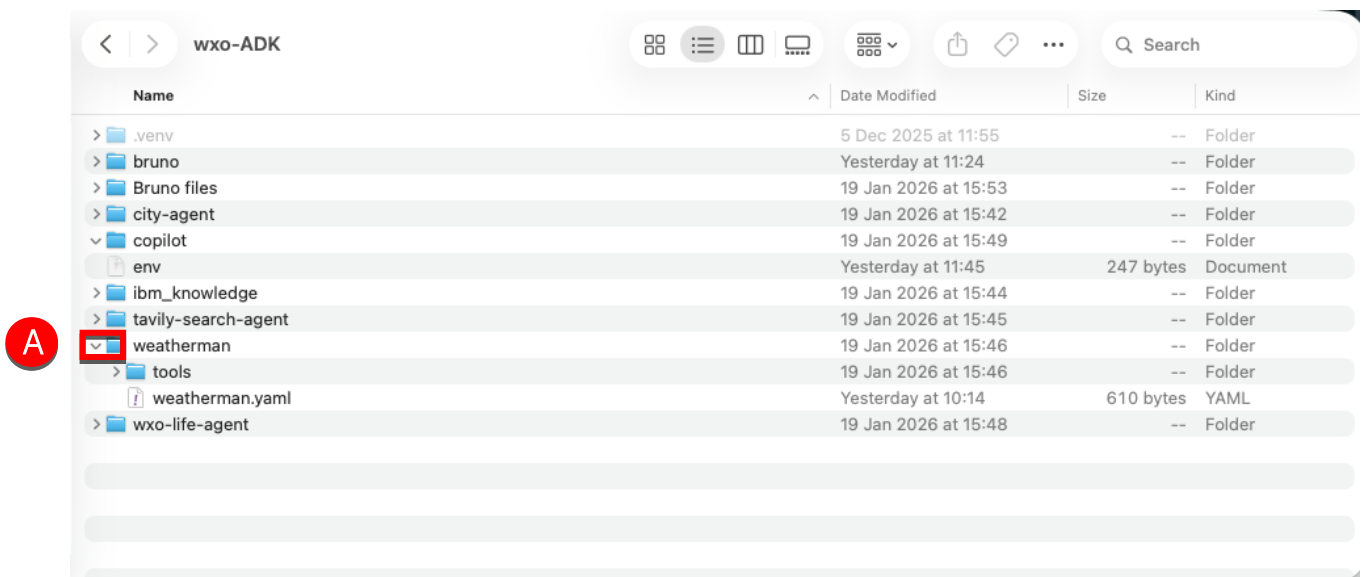
Creating and configuring a new agent

In this section you will create a tool that will enable agents to query for weather information (provided by open-meteo.com) to be used in responses to user queries. This tool will be built by importing the OpenAPI specification for the web service, eliminating the need to write any API-calling code manually. This is a requirement for creating the agent in a later section that will use this tool.

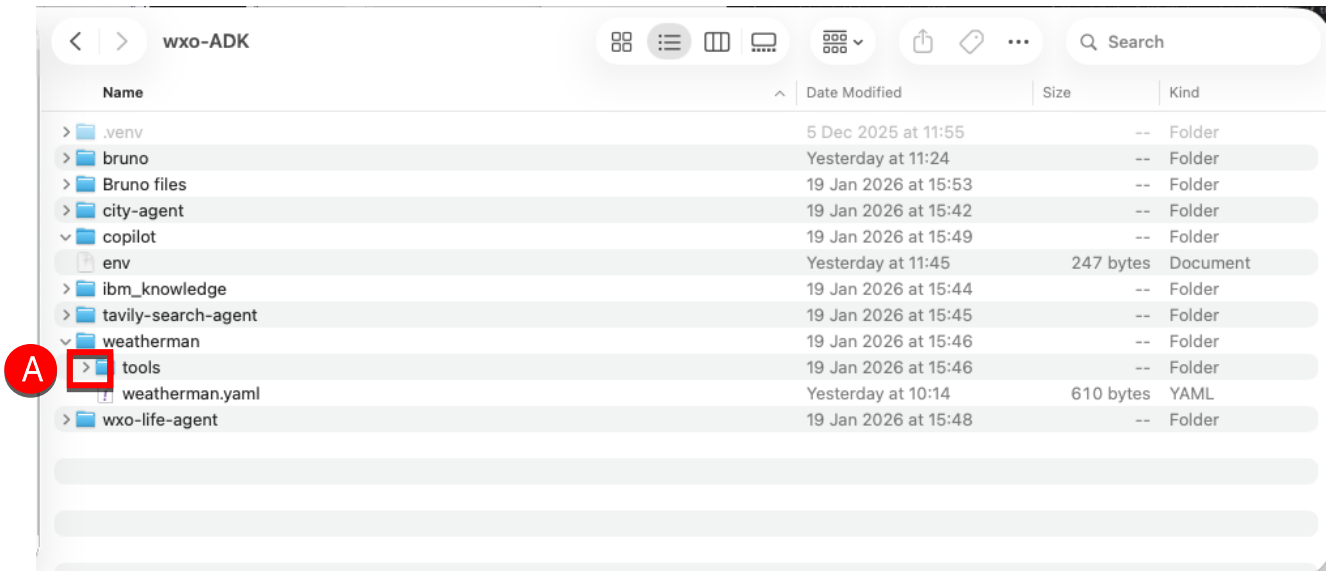
Creating a tool by importing a web-service

In this section we will review the OpenAPI fields required for importing a web-service as a tool in watsonx Orchestrate ADK and create a tool to retrieve weather details from.

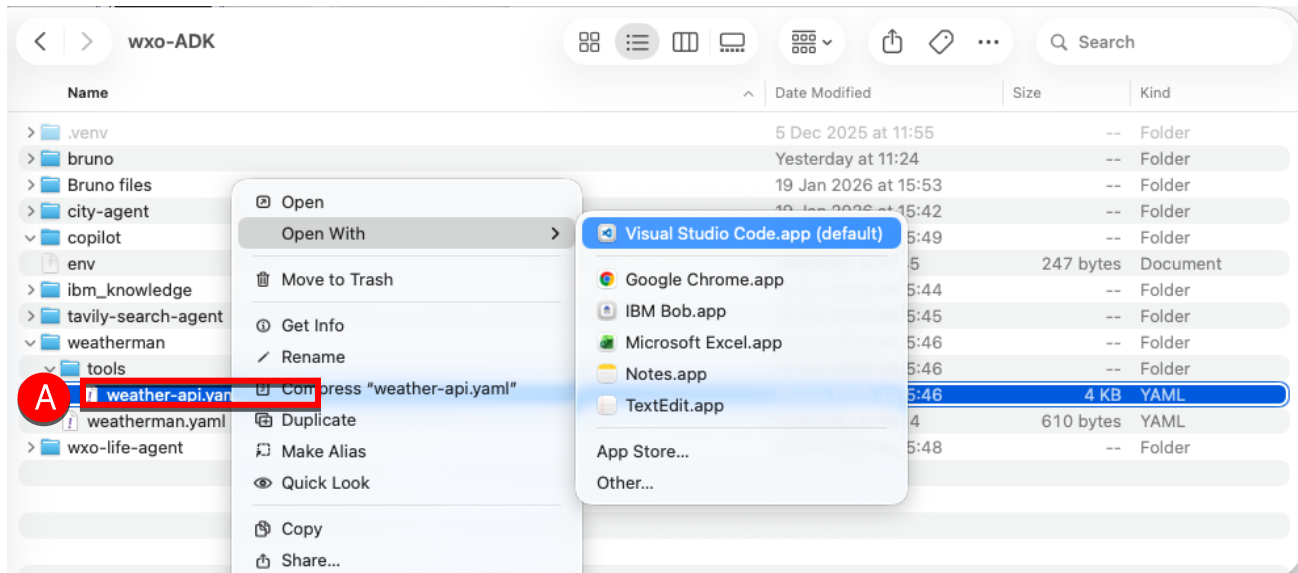
1. Navigate to your **wxo-ADK/weatherman** folder (A) to expand it.



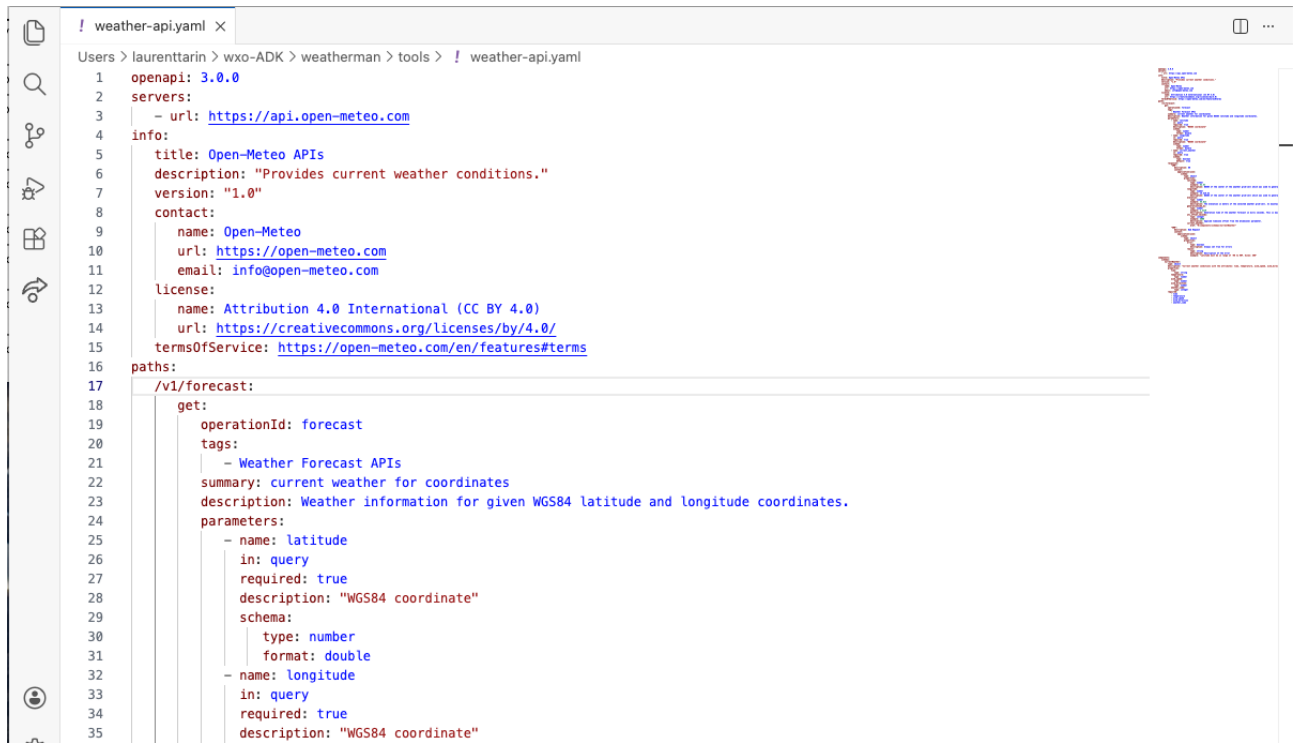
- Expand the **tools** folder (A).



- Edit the **weather-api.yaml** (A) with your preferred text editor.



4. Review the contents of the OpenAPI document.



```
1 openapi: 3.0.0
2 servers:
3   - url: https://api.open-meteo.com
4 info:
5   title: Open-Meteo APIs
6   description: "Provides current weather conditions."
7   version: "1.0"
8   contact:
9     name: Open-Meteo
10    url: https://open-meteo.com
11    email: info@open-meteo.com
12  license:
13    name: Attribution 4.0 International (CC BY 4.0)
14    url: https://creativecommons.org/licenses/by/4.0/
15  termsOfService: https://open-meteo.com/en/features#terms
16 paths:
17   /v1/forecast:
18     get:
19       operationId: forecast
20       tags:
21         - Weather Forecast APIs
22       summary: current weather for coordinates
23       description: Weather information for given WGS84 latitude and longitude coordinates.
24       parameters:
25         - name: latitude
26           in: query
27           required: true
28           description: "WGS84 coordinate"
29           schema:
30             type: number
31             format: double
32         - name: longitude
33           in: query
34           required: true
35           description: "WGS84 coordinate"
```

An OpenAPI document is a machine-readable description of an API, typically written in JSON or YAML format, that outlines the API's endpoints, parameters, responses, and other relevant details. It serves as a formal specification that helps developers understand how to interact with the API and can be used to generate documentation and client libraries.

On line 2, the servers element specifies the base URLs for the API, indicating where the API can be accessed. It allows developers to define multiple server URLs, which can be useful for different environments like development, testing, and production.

On line 16 the paths element defines the available endpoints (or resources) of an API and the operations (HTTP methods) that can be performed on those endpoints. Our example only has a GET operation, but each path can be specified with a unique identifier and can support multiple operations like GET, POST, PUT, and DELETE.

On line 19, the operationId provides a logical name for this particular definition, which will be used in the next step, when we're creating a tool from it.

- Enter the following command (A) into the terminal to create a tool using the API specification:

```
orchestrate tools import -k openapi -f ./weatherman/tools/weather-api.yaml
```

wxo-ADK — -zsh — 192x31

watsonx/ibm/granite-guardian-3-8b	The Granite model series is a family of IBM-trained, dense decoder-only models, which are particularly well-suited for generative tasks.
watsonx/meta-llama/llama-3-1-70b-gptq	Llama 3.1 is an auto-regressive language model that uses an optimized transformer architecture. The tuned versions use supervised fine-tuning (SFT) and reinforcement learning with human feedback (RLHF) to align with human preferences for helpfulness and safety.
watsonx/meta-llama/llama-3-1-8b	Llama-3-1-8b is an auto-regressive language model that uses an optimized transformer architecture.
watsonx/meta-llama/llama-3-2-11b-vision-instruct	Llama-3-2-11b-vision-instruct is an auto-regressive language model that uses an optimized transformer architecture.
watsonx/meta-llama/llama-3-3-70b-instruct	This version of Llama-3.3-70b-instruct is also the FP8 quantized version of the original FP16 weights.
watsonx/meta-llama/llama-4-maverick-17b-128e-instruct-fp8	Llama 4 Maverick, a 17 billion active parameter model with 128 experts.
watsonx/meta-llama/llama-guard-3-11b-vision	Llama-guard-3-11b-vision is an auto-regressive language model that uses an optimized transformer architecture.
watsonx/mistral-large-2512	Mistral Large 3, is a state-of-the-art, open-weight, general-purpose multimodal model with a granular Mixture-of-Experts architecture. It features 41B active parameters and 675B total parameters.
watsonx/mistralai/mistral-medium-2505	Mistral Medium 3 (25.05) is our latest iteration of the Mistral Medium model family.
watsonx/mistralai/mistral-small-3-1-24b-instruct-2503	This model is an instruction-finetuned version of: Mistral-Small-3.1-24B-Base-2503.
watsonx/openai/gpt-oss-120b	openai/gpt-oss-120b is an OpenAI's open-weight models designed for powerful reasoning, agentic tasks, and versatile developer use cases.

* indicates a supported and preferred model from watsonx
 * indicates a model from a custom provider

ADK orchestrate tools import -k openapi -f ./weatherman/tools/weather-api.yaml A

You should see a message that the tool has been successfully imported into your watsonx Orchestrate ADK environment.

Instructions for using the ADK command line interface are provided for each step in this lab. You can enter the command below to explore the CLI commands.

```
orchestrate --help
```

wxo-ADK — -zsh — 161x39

```
ADK > orchestrate tools import -k openapi -f ./weatherman/tools/weather-api.yaml
[INFO] - Tool 'forecast' imported successfully
ADK > orchestrate --help
```

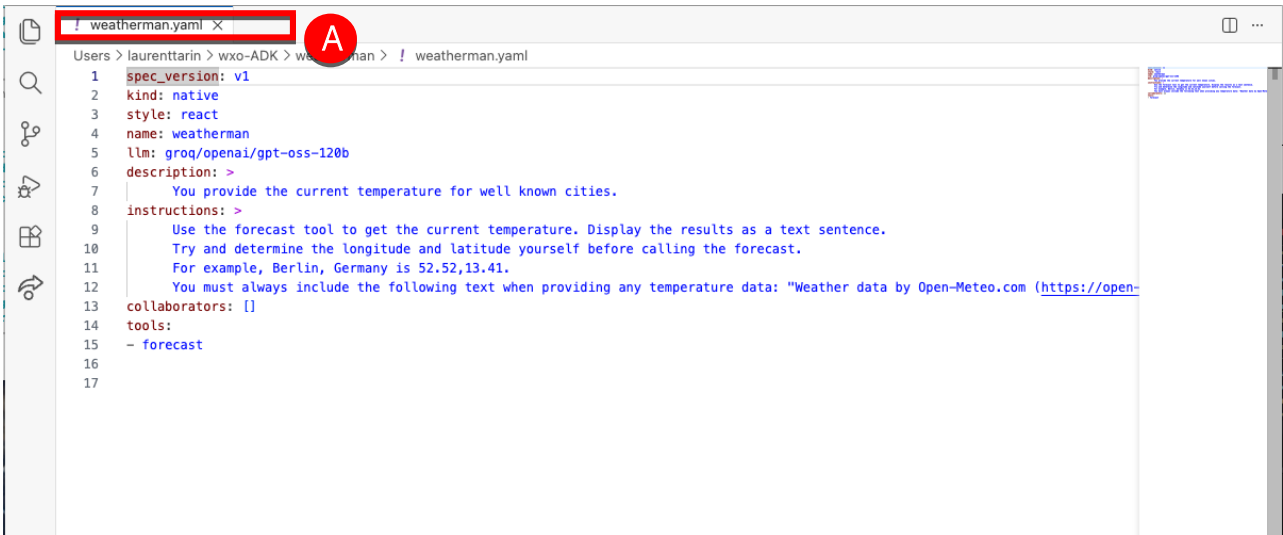
Usage: `orchestrate [OPTIONS] COMMAND [ARGS]...`

Options	
<code>--version</code>	Show the installed version of the ADK and Developer Edition Tags
<code>--debug</code>	Enable debug mode
<code>--install-completion</code>	Install completion for the current shell.
<code>--show-completion</code>	Show completion for the current shell, to copy it or customize the installation.
<code>--help</code>	Show this message and exit.

Commands	
<code>env</code>	Add, remove, or select the activate env other commands will interact with (either your local server or a production instance)
<code>agents</code>	Interact with the agents in your active env
<code>tools</code>	Interact with the tools in your active env
<code>toolkits</code>	Interact with the toolkits in your active env
<code>knowledge-bases</code>	Upload knowledge your agents can search through to your active env
<code>connections</code>	Interact with the agents in your active env
<code>voice-configs</code>	Configure voice providers to enable voice interaction with your agents
<code>server</code>	Manipulate your local Orchestrate Developer Edition server [requires entitlement]
<code>chat</code>	Launch the chat ui and CLI chat interface for your local Developer Edition server [requires entitlement]
<code>models</code>	List the available large language models (llms) that can be used in your agent definitions
<code>channels</code>	Configure channels where your agent can exist on (such as embedded webchat, whatsapp, etc.)
<code>phone</code>	Manage phone configurations (global resources that can be attached to multiple agents)
<code>evaluations</code>	Evaluate the performance of your agents in your active env
<code>settings</code>	Configure the settings for your active env
<code>partners</code>	Generate a well-structured, submission-ready agent artifact package for partner-built agents

Agent creation

1. Edit the **weatherman.yaml** file (A) and review the definition of the agent.

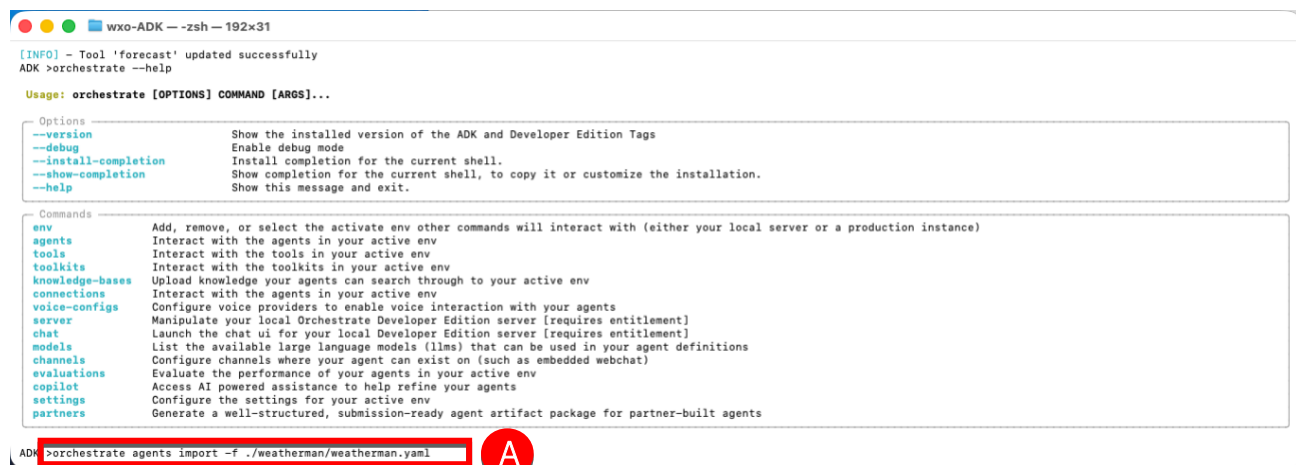


```
1 spec_version: v1
2 kind: native
3 style: react
4 name: weatherman
5 llm: groq/openai/gpt-oss-120b
6 description: >
7   You provide the current temperature for well known cities.
8 instructions: >
9   Use the forecast tool to get the current temperature. Display the results as a text sentence.
10  Try and determine the longitude and latitude yourself before calling the forecast.
11  For example, Berlin, Germany is 52.52,13.41.
12  You must always include the following text when providing any temperature data: "Weather data by Open-Meteo.com (https://open-meteo.com)".
13 collaborators: []
14 tools:
15   - forecast
16
17
```

This YAML file defines the agent using the ADK specification for agents. It defines the agent description, the instructions which define the agent's behavior, and the LLM model it will use for planning and reasoning. On line 14, under the tools element there is a reference to the id of the tool we imported in the former step – forecast, which the agent will be able to use.

2. Enter the following command (A) into the terminal to create the agent:

```
orchestrate agents import -f ./weatherman/weatherman.yaml
```



```
wxo-ADK — zsh — 192x31
[INFO] - Tool 'forecast' updated successfully
ADK > orchestrate --help

Usage: orchestrate [OPTIONS] COMMAND [ARGS]...

Options
  --version          Show the installed version of the ADK and Developer Edition Tags
  --debug            Enable debug mode
  --install-completion  Install completion for the current shell.
  --show-completion  Show completion for the current shell, to copy it or customize the installation.
  --help            Show this message and exit.

Commands
  env                Add, remove, or select the active env other commands will interact with (either your local server or a production instance)
  agents             Interact with the agents in your active env
  tools              Interact with the tools in your active env
  toolkits           Interact with the toolkits in your active env
  knowledge-bases    Upload knowledge your agents can search through to your active env
  connections        Interact with the agents in your active env
  voice-configs      Configure voice providers to enable voice interaction with your agents
  server             Manipulate your local Orchestrate Developer Edition server [requires entitlement]
  chat               Launch the chat ui for your local Developer Edition server [requires entitlement]
  models             List the available large language models (llms) that can be used in your agent definitions
  channels            Configure channels where your agent can exist on (such as embedded webchat)
  evaluations         Evaluate the performance of your agents in your active env
  copilot            Access AI powered assistance to help refine your agents
  settings           Configure the settings for your active env
  partners           Generate a well-structured, submission-ready agent artifact package for partner-built agents

ADK > orchestrate agents import -f ./weatherman/weatherman.yaml
```

You should see a message that the agent has been successfully imported into your watsonx Orchestrate ADK environment.

Congratulations, this concludes the creation of your agent.

Listing available agents using the CLI

In this section you will be using the CLI to view the list of agents available in your installation of watsonx Orchestrate ADK.

1. To list agents imported and available within watsonx Orchestrate run the following command (A):

```
orchestrate agents list
```



```
wxo-ADK -- zsh -- 192x31

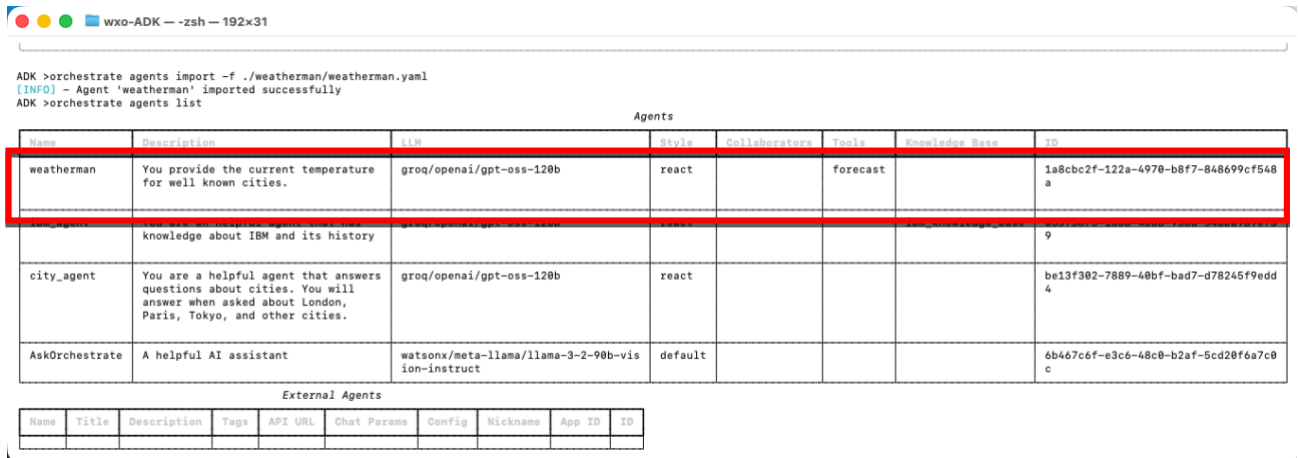
Usage: orchestrate [OPTIONS] COMMAND [ARGS]...

Options
  --version          Show the installed version of the ADK and Developer Edition Tags
  --debug            Enable debug mode
  --install-completion Install completion for the current shell.
  --show-completion  Show completion for the current shell, to copy it or customize the installation.
  --help            Show this message and exit.

Commands
  env          Add, remove, or select the activate env other commands will interact with (either your local server or a production instance)
  agents       Interact with the agents in your active env
  tools        Interact with the tools in your active env
  toolkits     Interact with the toolkits in your active env
  knowledge-bases Upload knowledge your agents can search through to your active env
  connections  Interact with the agents in your active env
  voice-configs Configure voice providers to enable voice interaction with your agents
  server       Manipulate your local Orchestrate Developer Edition server [requires entitlement]
  chat         Launch the chat ui for your local Developer Edition server [requires entitlement]
  models       List the available large language models (llms) that can be used in your agent definitions
  channels     Configure channels where your agent can exist on (such as embedded webchat)
  evaluations  Evaluate the performance of your agents in your active env
  copilot      Access AI powered assistance to help refine your agents
  settings     Configure the settings for your active env
  partners     Generate a well-structured, submission-ready agent artifact package for partner-built agents

ADK >orchestrate agents import -f ./weatherman/weatherman.yaml
[INFO] - Agent 'weatherman' imported successfully
ADK >orchestrate agents list
```

2. The results should resemble the example below, showing the **weatherman** agent:



```
wxo-ADK -- zsh -- 192x31

ADK >orchestrate agents import -f ./weatherman/weatherman.yaml
[INFO] - Agent 'weatherman' imported successfully
ADK >orchestrate agents list
```

Agents							
Name	Description	LLM	Style	Collaborators	Tools	Knowledge Base	ID
weatherman	You provide the current temperature for well known cities.	groq/openai/gpt-oss-120b	react		forecast		1a8cbc2f-122a-4970-b8f7-848699cf548a
	knowledge about IBM and its history						9
city_agent	You are a helpful agent that answers questions about cities. You will answer when asked about London, Paris, Tokyo, and other cities.	groq/openai/gpt-oss-120b	react				be13f302-7889-40bf-bad7-d78245f9edd4
AskOrchestrate	A helpful AI assistant	watsonx/meta-llama/llama-3-2-90b-vision-instruct	default				6b467c6f-e3c6-48c0-b2af-5cd20f6a7c0c

External Agents									
Name	Title	Description	Tags	API URL	Chat Params	Config	Nickname	App ID	ID

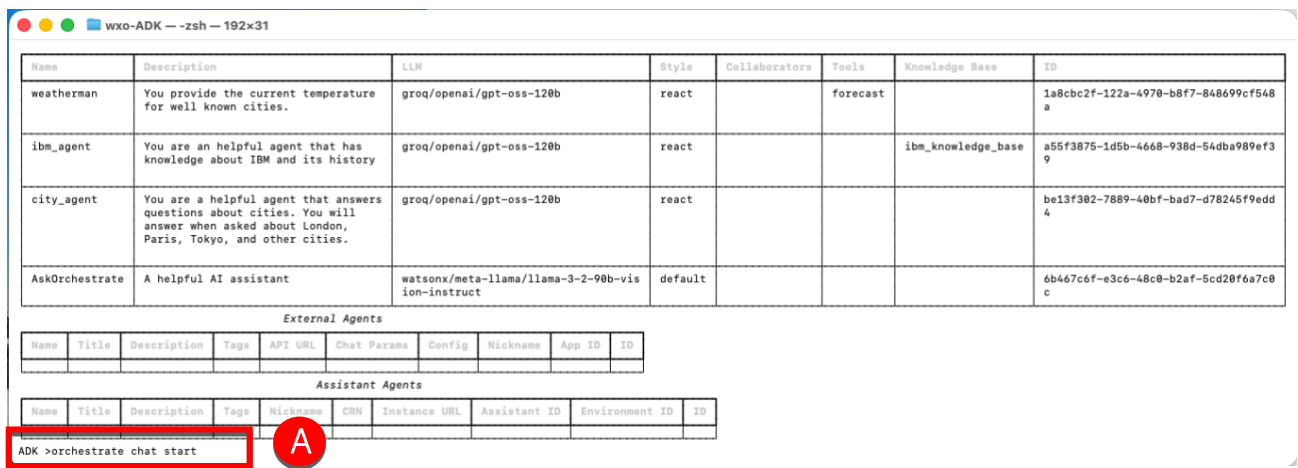
Testing the weatherman agent

In this section you will start the chat interface, test the agent and validate that it's using the tool created earlier to generate responses effectively, enabling the agent to include live weather data within its generated response.

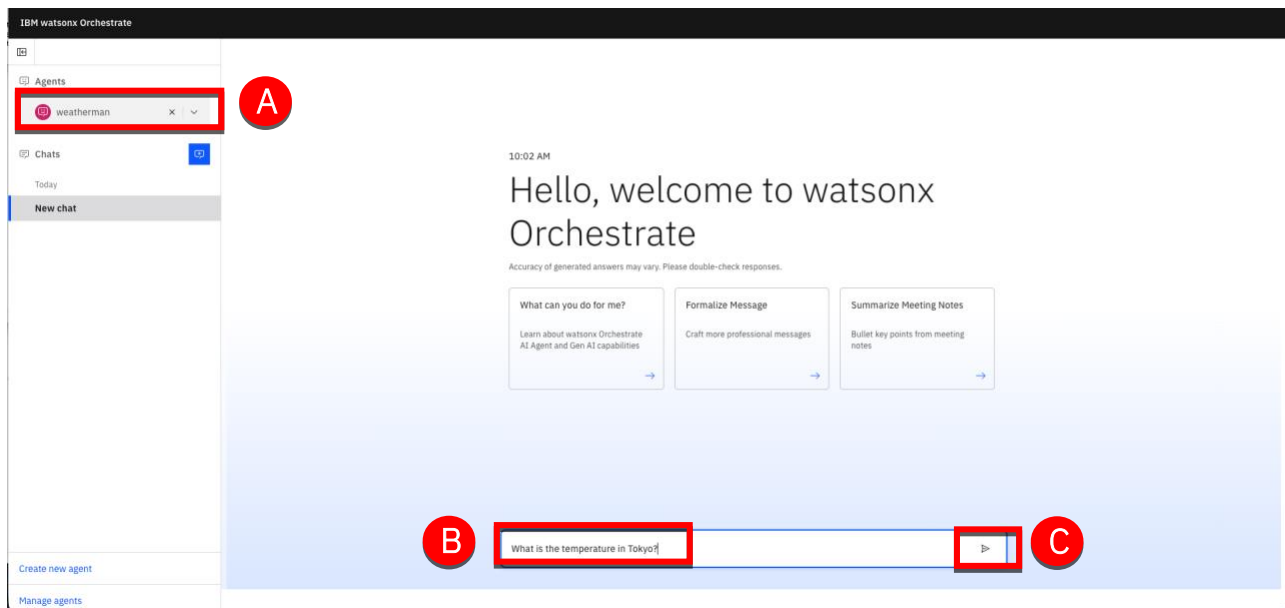
Agent and tool testing

1. Enter the following command into the terminal (A) to start the chat server.

```
orchestrate chat start
```



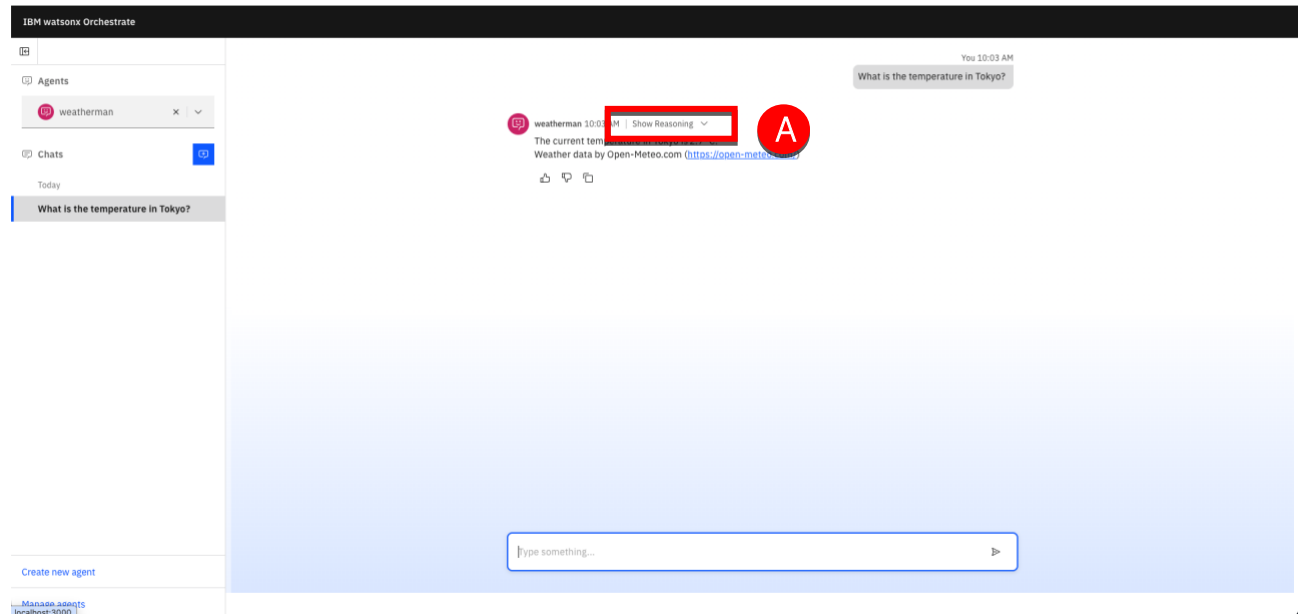
2. Wait for the server to load and when the browser opens, make sure to select the **weatherman** agent in the Agents dropdown (A), then in the chat, enter "What is the temperature in Tokyo?" into the chat input (B) and click the submit icon (C).



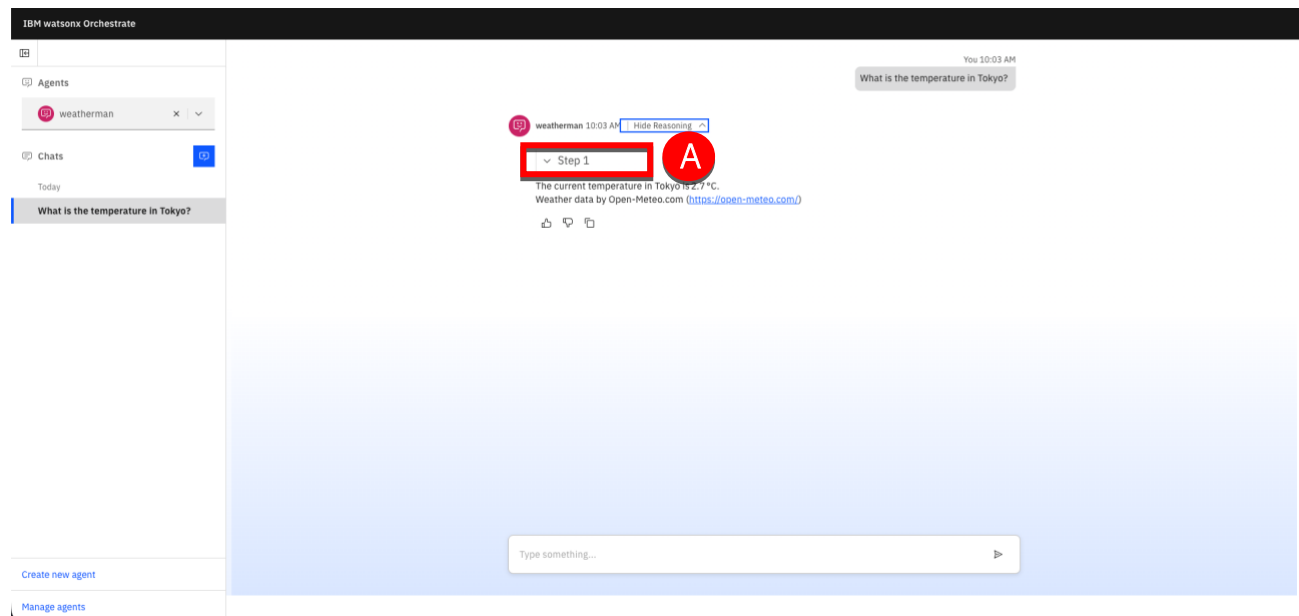
Wait for the agent to reason about how to perform the action and then wait for the response from the agent. Observe that the agent first began planning & reasoning and then responded to the question using live data from the web-service (service provided by open-meteo.com).

If you encounter an error, refer to the troubleshooting guide at the end of this document.

3. Expand the **Show Reasoning** panel by clicking the expander icon (A).



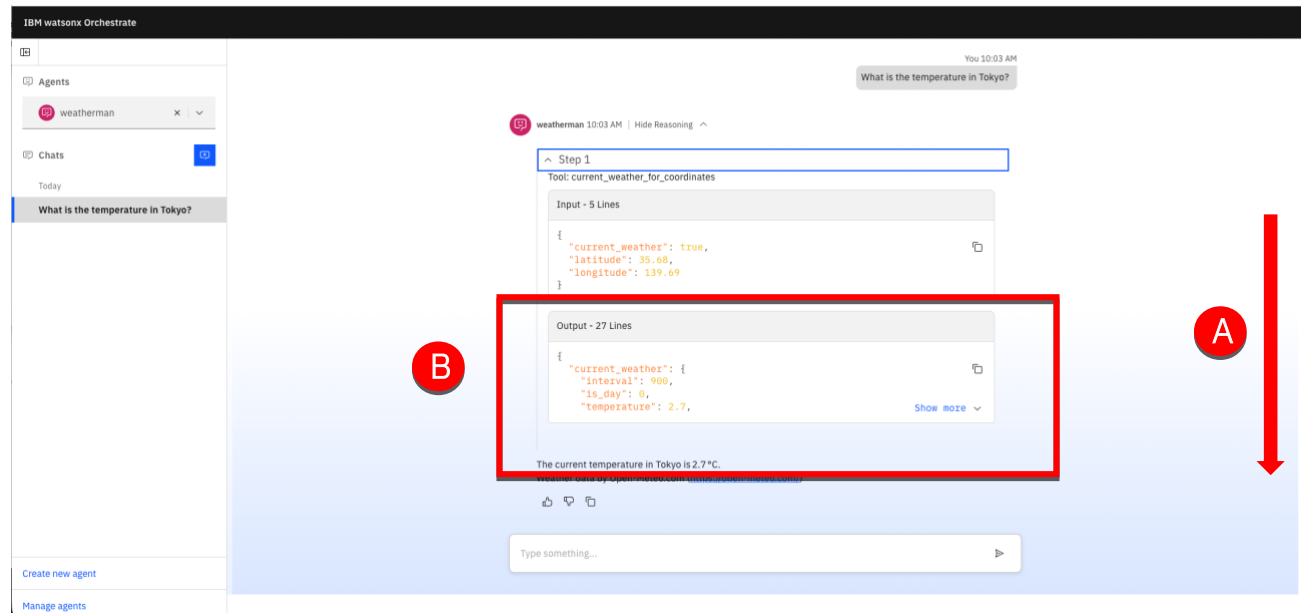
4. Click to expand **Step 1** (A).



Observe how the agent understands the format of the information required by the web-service to return the current temperature for the location of Tokyo and formats it according to the OpenAPI

specification that was provided earlier. The Llama vision model used by the agent is able to infer the longitude and latitude for Tokyo.

5. Scroll down (A) to the **Output** panel (B):



Observe the response from the web-service call and how the agent was able to infer the right field to take temperature result from, generate the response text and format the response for the user.

Summary

In this lab you created a tool by importing an OpenAPI web-service definition and then created an agent that was able to provide live temperature data in response to user queries. The agent was able to adapt to the users request and interpret the information provided from the service and format it as a natural language response using the reasoning power of the LLM.

Next steps

Congratulations. You have completed this watsonx Orchestrate L4 lab. Continue to build on the knowledge acquired in this lab, in the following labs you will be building agents based on knowledge, and more complex agents that use Python tools that collaborate with each other.

Appendix: Troubleshooting

System unresponsive

If your ADK environment becomes unresponsive use the command below to stop all running containers.

```
docker stop $(docker ps -q)
```

Enter the following command into the terminal to reset your server, then press Enter.

```
orchestrate server reset
```

Enter the following command into the terminal to restart your server, then press Enter.

```
orchestrate server start -l -e env
```

Enter the following command into the terminal (A) to start the chat server.

```
orchestrate chat start
```

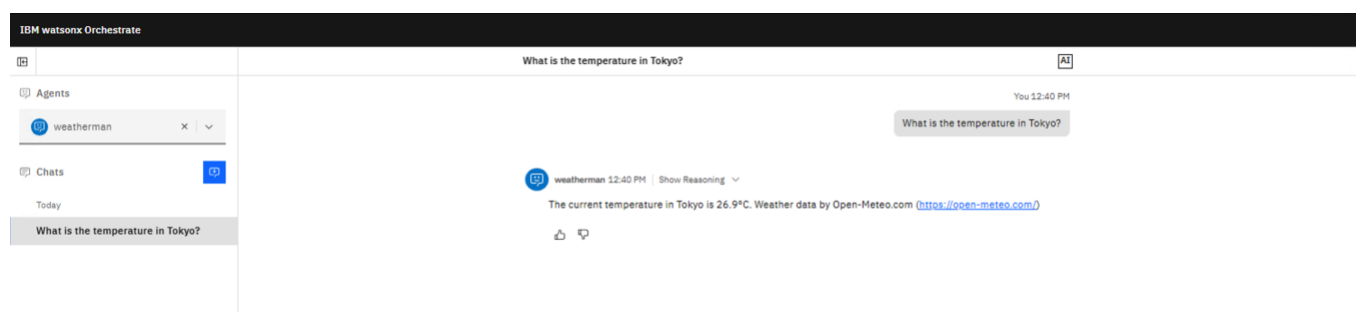
You will need to repeat any commands to create tools and agents.

LLM Error

If you receive an error when using your agent (that worked previously), your API key may have been deleted. You will need to recreate the API key from your watsonx Orchestrate instance, update your env file, and restart the server.

Weatherman error

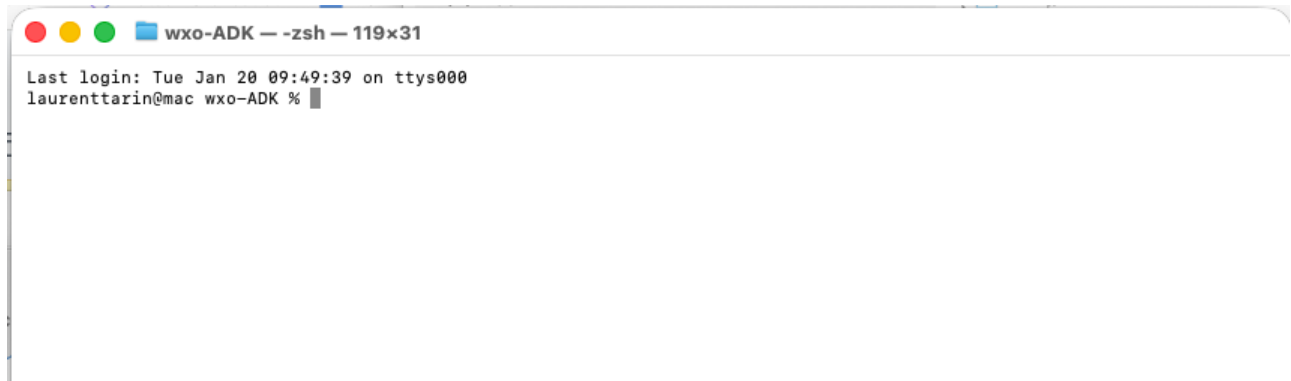
If your weatherman agent does not respond as expected, click the browser refresh button then click the new chat icon (A) and repeat the question. The LLM used by the agent can sometimes respond unpredictably, especially under heavy load.



Appendix: Reset your environment

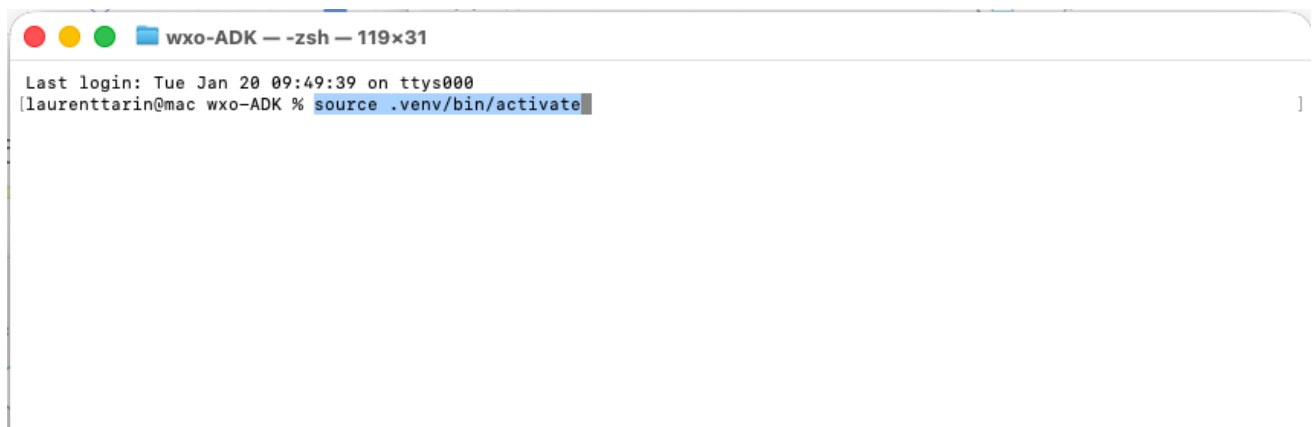
These steps will remove any existing tools and agents from your environment and restart the local watsonx Orchestrate server in readiness to start the lab.

1. Open a Terminal window on your **wxO-ADK** folder (A).



2. Enter the following command into the terminal to start your virtual environment:

```
source .venv/bin/activate
```



3. Check that the prompt starts with (.venv) (A) indicating that you are in your virtual environment.

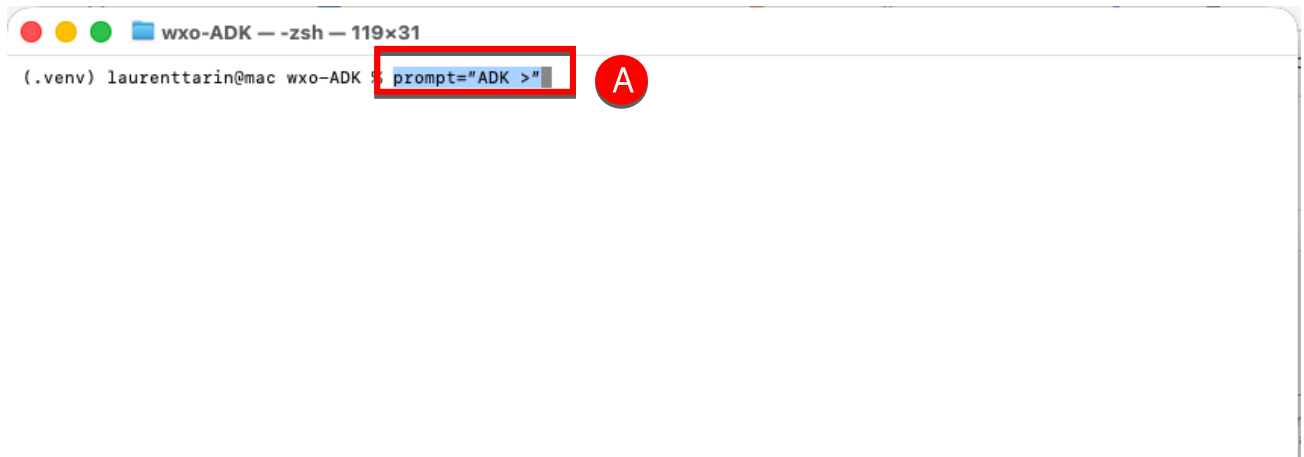


A terminal window titled 'wxo-ADK --zsh-- 119x31'. The output shows the last login time and the execution of the 'source .venv/bin/activate' command, which changes the prompt to '(.venv) laurenttarin@mac wxo-ADK %'.

```
Last login: Tue Jan 20 09:49:39 on ttys000
[laurenttarin@mac wxo-ADK % source .venv/bin/activate
(.venv) laurenttarin@mac wxo-ADK %
```

4. Enter the following command into the terminal to improve the command visibility, then press Enter:

```
MS Windows users: function prompt{"ADK>"}
Mac users: prompt="ADK >"
```



A terminal window titled 'wxo-ADK --zsh-- 119x31'. The prompt is '(.venv) laurenttarin@mac wxo-ADK \$'. The command 'prompt="ADK >"' is being entered and is highlighted with a red box. A red circle with the letter 'A' is next to the command.

```
(.venv) laurenttarin@mac wxo-ADK $ prompt="ADK >"
```

5. Enter the following command into the terminal to stop your chat service, then press Enter.

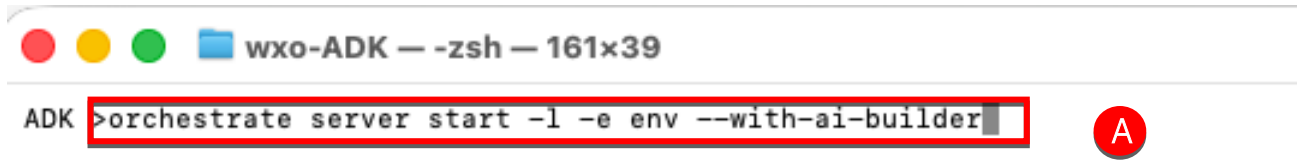
```
orchestrate chat stop
```

6. Enter the following command into the terminal to reset your server, then press Enter.

```
orchestrate server reset
```

7. Enter the following command into the terminal to start the server with your env configuration, then press Enter.

```
orchestrate server start -l -e env --with-ai-builder
```



Note: Some services may fail to start on the first attempt, and you may encounter issues connecting to the docker repository. Retry the start command in this situation, depending on the volume of requests, it can take several attempts to connect.

When startup is complete the terminal output will resemble the example below, restart can take 2-3 minutes to complete. The local server will be ready when you will see this **INFO** in the terminal screen (A):

```
wxo-ADK -- -zsh -- 119x31

Skipping already applied migration: 20251001.sql
Skipping already applied migration: 20251006.sql
Skipping already applied migration: 20251007.sql
Skipping already applied migration: 20251008.sql
Skipping already applied migration: 20251009.sql
Skipping already applied migration: 20251022.sql
Skipping already applied migration: 20251023.sql
Skipping already applied migration: 20251029.sql
Skipping already applied migration: 20251101.sql
Skipping already applied migration: 20251103.sql
Skipping already applied migration: 20251104.sql
Skipping already applied migration: 20251107.sql
Skipping already applied migration: 20251110.sql
Skipping already applied migration: 20251111.sql
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Waiting for orchestrate server to be fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the
UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, pa
ssword: orchestrate
ADK
```



You are now ready to work with watsonx Orchestrate ADK.

